MANIPAL
ACADEMY of HIGHER EDUCATION
(Deemed to be University under Section 3 of the UGC Act, 1956)

MANIPAL ACADEMY OF HIGHER EDUCATION

BENGALURU, INDIA

Insurance Fraud Claim Detection System Using Google Gemini AI

A project report submitted in partial fulfilment of the academic requirements for the Bachelor of Technology programme.

SUBMITTED BY

Student Name	**Vipul Srivastava**
Registration Number	2428010117
Section	B
Project Guide	Dr. Manoj Kumar Sharma

Manipal Academy of Higher Education

TABLE OF CONTENTS

No.	Chapter	Page
1.	Introduction	2
2.	Objectives	2
3.	Hardware And Software Requirements	2
4.	System Architecture And Working	3
5.	Flowchart And Block Diagram	3
6.	Module Description	3
7.	Implementation And Technologies	4
8.	Results And Output	4
9.	Advantages And Limitations	5
10.	Future Scope	5
11.	Conclusion	5
12.	References	6

1. INTRODUCTION

Insurance claim review checks coverage, incident details, submitted amounts and supporting evidence. Fraud may involve staged events, inflated losses or false documents, but an unusual claim is not proof of deception. Automated tools can help prioritize investigation; people must verify evidence and make consequential decisions.

This project is a client-side React and TypeScript single-page application for vehicle and health claim examples. It validates selected fields, applies fixed local scoring rules and optionally asks Google Gemini 2.5 Flash to produce explanatory text. Gemini is an enrichment step: it does not change the local score or tier. The repository contains no custom API server, claims database, authentication system or trained fraud model.

2. OBJECTIVES

- Accept vehicle and health claim details in separate browser forms.
- Validate selected required, numeric and date fields before analysis.
- Calculate a transparent rule-based score, LOW/MEDIUM/HIGH tier and triggered factors.
- Offer optional structured Gemini narrative and follow-up Copilot assistance.
- Retain a baseline-only workflow when Gemini is not configured.
- Demonstrate the workflow with a responsive UI and sample presets.

Training a model, checking external records, storing cases and producing calibrated fraud probabilities are outside the implemented scope.

3. HARDWARE AND SOFTWARE REQUIREMENTS

Area	Practical requirement
Hardware	Modern dual-core computer; 4 GB RAM minimum estimate; 1280×720 display; 2 GB free storage estimate.
Operating system	Windows, macOS or Linux with a supported Node.js environment; Docker image uses Linux Alpine.
Application stack	React 19, TypeScript, Vite 6, Tailwind CSS 4 and lucide-react.
AI integration	Optional @google/genai SDK and Google Gemini 2.5 Flash API key; internet required for requests.
Build and deployment	Node.js 20 in Docker builder; npm scripts; Nginx serves static output; Vercel SPA rewrite is configured.
Persistence	No database. Claims/results remain in component state; browser localStorage can hold the Gemini key.

Hardware figures are practical estimates, not measured guarantees. The manifest lists Express, dotenv and Motion dependencies, but the inspected source does not implement an Express server or use those packages for the claimed runtime features.

4. SYSTEM ARCHITECTURE AND WORKING

The browser hosts the entire application. `main.tsx` mounts React; `App.tsx` manages navigation; the dashboard selects a claim type; the matching form validates and submits typed data to `ClaimAnalysis`. The page calls `analyzeClaim` in `src/services/prediction.ts`, which calculates the authoritative local result. If a key is available, browser code in

`src/services/gemini.ts` makes a direct request to Google. There is no Veridex backend between the browser and Gemini.

Vehicle and health scores start at 20 and 18 respectively, apply manually selected thresholds, are clamped to 8–96 and 9–95, then map to LOW (<35), MEDIUM (35–64) or HIGH (≥65). Claim and result state is not persisted. Docker builds static files and Nginx serves them; the Docker image does not start Express.

5. FLOWCHART AND BLOCK DIAGRAM

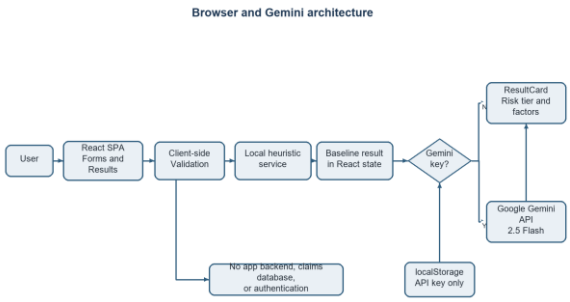


Figure 1. Browser, local scoring and optional Gemini architecture

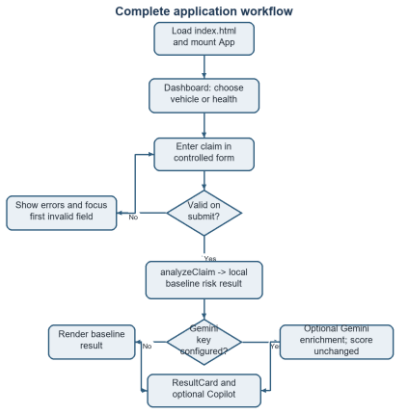


Figure 2. Claim review flow: validate, score and optionally enrich

The Mermaid definitions for the architecture, application flow, claim scoring, Gemini integration and data flow are supplied as separate `.mmd` files in this report folder.

6. MODULE DESCRIPTION

Module	Responsibility
App and pages	Navigation, dashboard, forms, analysis state and system information.
Car/health forms	Controlled inputs, demo presets, validation feedback and typed submission.
Validation utility	Returns field-level errors for selected required, range and date checks.
Prediction service	Applies the separate deterministic vehicle and health rules.
Gemini service and key utility	Resolves key, builds structured request, parses response and falls back on error.
Result/settings/Copilot UI	Shows local result and optional AI text; manages key testing and follow-up prompts.

7. IMPLEMENTATION AND TECHNOLOGIES

React function components and hooks manage the interface and form state. TypeScript models the claim variants and results. Vite builds the SPA; Tailwind supplies styles and Lucide React supplies icons. Form validators check input in the browser but are not server security controls or coverage checks.

Rule group	Implemented examples
Vehicle	Claim/premium ratio >50: +26; late incident without police report: +22; recent tenure and large loss: +20; minor damage with high injury claim: +18. Long tenure and multiple witnesses with a report can subtract points.
Health	Loss within 14 days of policy start: +30; claim/premium ratio >40: +24; selected watchlist segment: +18; mild severity with high claim: +22. Preferred-risk segment can subtract points.

Rules use only a subset of form fields. `confidenceScore` and the reference suffix use `Math.random`; the former is not a validated confidence estimate. A zero premium can pass validation and be replaced by 1 in scoring, inflating the ratio. Gemini receives selected claim data and baseline flags, requests JSON at low temperature, and requires an executive summary and pattern label. Parsing is limited; generated text is not independently verified, and its recommended action is not rendered. The local score remains authoritative.

The settings utility checks localStorage and Vite environment values, while `vite.config.ts` defines `process.env.GEMINI_API_KEY`; the utility reads `import.meta.env.GEMINI_API_KEY`, so the names are inconsistent. Browser-stored or bundled keys are exposed to browser code and are not server-side secrets.

8. RESULTS AND OUTPUT

The supplied genuine screenshots show the System Spec page and Gemini configuration modal of the running application. They demonstrate informational and settings views, not a scored claim result. The application can display risk tier, percentage-style score, factors, recommendation, claim summary and—when configured—Gemini narrative, pattern, inconsistencies and checklist. No accuracy, precision, recall, false-positive rate or throughput results are claimed because no evaluation dataset or benchmark is included.

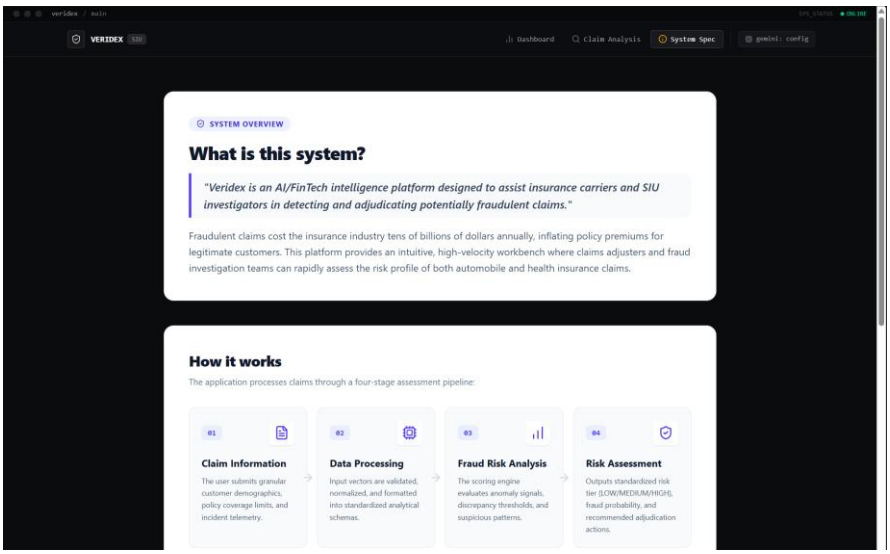


Figure 3. System specification page

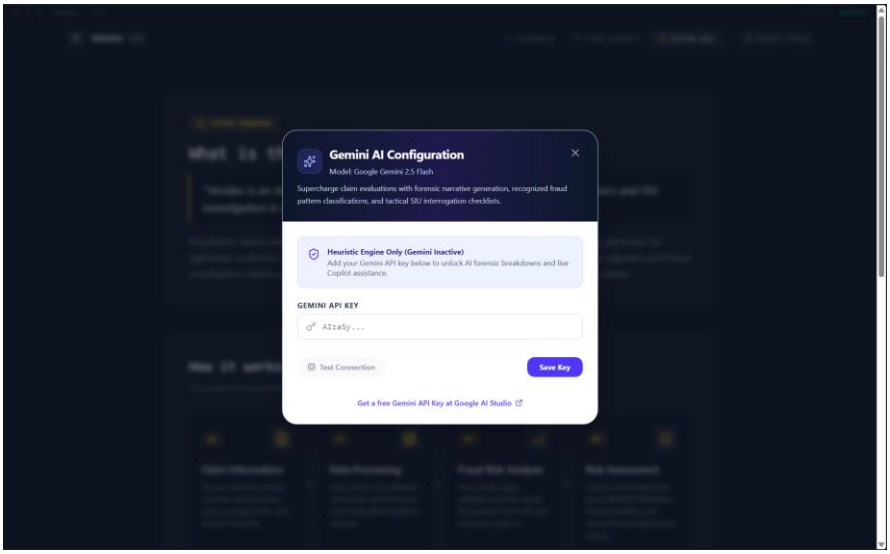


Figure 4. Gemini configuration modal

9. ADVANTAGES AND LIMITATIONS

Advantages: the rules and triggered factors are inspectable; the two claim types have separate fields; baseline scoring works without Gemini; the JSON response shape constrains AI output; presets support demonstrations; and static hosting is straightforward.

Limitations: this is a manually selected heuristic, not trained ML or calibrated probability. There is no backend, database, authentication, access control, audit trail or insurer data integration. Runtime validation of Gemini output is limited; failures have no retry/backoff; some collected fields do not affect scoring. Browser localStorage/build configuration exposes the API key. Prompts send claim information to an external service, with no consent, redaction or retention controls. Do not use real sensitive claims or make operational decisions from this demo without an approved security, privacy and human-review design.

10. FUTURE SCOPE

- Move Gemini calls to an authenticated backend and protect provider secrets.
- Add role-based access, secure case storage, audit history, consent and data minimization.
- Evaluate rules/models with representative labeled data, calibration, bias checks and documented error costs.
- Validate all AI fields and evidence, expose uncertainty, add timeouts, limits, monitoring and retries.
- Add accessibility, automated tests, and human review before any consequential decision.

11. CONCLUSION

The project demonstrates a browser workflow for entering vehicle or health claims, validating selected fields, applying explainable heuristics and presenting a review-oriented summary. Optional Gemini 2.5 Flash text enriches the explanation while leaving the score unchanged. The separation of local scoring from generated narrative is clear, but the rules and scores have not been empirically validated. This prototype is suitable for an academic demonstration; real use would require secure services, privacy controls, auditable human review and rigorous evaluation.

12. REFERENCES

[1] Google AI for Developers. Gemini API: Generate content and structured outputs. https://ai.google.dev/gemini-api/docs (accessed 1 November 2026).

[2] React. Managing State. https://react.dev/learn/managing-state (accessed 1 November 2026).

[3] Microsoft. TypeScript Handbook. https://www.typescriptlang.org/docs/handbook/ (accessed 1 November 2026).

[4] Vite. Env Variables and Modes. https://vite.dev/guide/env-and-mode (accessed 1 November 2026).

[5] Tailwind CSS. Install with Vite. https://tailwindcss.com/docs/installation/using-vite (accessed 1 November 2026).

[6] Docker. Dockerfile reference. https://docs.docker.com/reference/dockerfile/ (accessed 1 November 2026).

[7] Nginx. Beginner's Guide. https://nginx.org/en/docs/beginners_guide.html (accessed 1 November 2026).

[8] Debener et al. Detecting insurance fraud using supervised and unsupervised machine learning. Journal of Risk and Insurance, 90(3), 2023. https://doi.org/10.1111/jori.12427.

[9] Schrijver et al. Automobile insurance fraud detection using data mining: a systematic literature review. Intelligent Systems with Applications, 21, 2024. https://doi.org/10.1016/j.iswa.2024.200340.

[10] Project source: `src/services/prediction.ts`, `gemini.ts`, `geminiKey.ts`, `utils/validation.ts`, `src/types.ts`, components and root deployment configuration.